

Resolve360

Stop chasing. Start resolving.

A governed Collections 360 with plain-English Ask and an affordability-first Next Best Action that changes the moment a call ends.



Collections 360

one customer, one trusted ID



Ask

answers that show their working



Next Best Action

risk + ability to pay



Governance

built into every layer

Team Hack It · Collections Hackathon · Design submission, 2 October 2026

Educational prototype on fully synthetic data. Not a system of, or endorsed by, the industry partner.

One customer. Nine systems. No single story.

CARDS

Visa 28 days past due,
\$452 overdue

LOANS

Personal loan 34 days
past due

DEPOSITS

No payroll deposit for 29
days

COLLECTIONS

Promise of \$1,000 broken
on 15 Sep

CRM

Hourly-wage customer
since 2016

CONTACT HISTORY

SMS reminder delivered,
no response

AGENT NOTES

Asked about options,
sounded anxious

CALL TRANSCRIPTS

"Shifts at the plant got
cut in August"

BUREAU

Two new credit inquiries



Fragmented

The agent toggles between systems. The hardship clue is buried in a call transcript.



Slow insight

"Who will break a promise this week?" becomes a data request that takes days.



Generic pressure

He gets the same reminders and calls as a customer who simply won't pay.

R. Mitchell, a synthetic customer from our mock data. The two highlighted clues explain everything, and no one sees them together.

Can't pay is not won't pay

Collections systems answer "who should we chase?". The better question, and the one the brief's own data can answer, is "what can this customer pay, and when?"

Today: who should we chase?

- Rank customers by risk
- Push hardest on the riskiest
- Same treatment whether they can't or won't pay
- Result: broken promises, rolls to worse buckets, fairness risk

Resolve360: what can they pay, and when?

- Risk plus ability to pay from salary credits
- Plan amount the customer can keep
- Contact and due date timed to the next payday
- Result: kept promises, fair treatment, vulnerable customers protected

Resolve360: one working slice, five differentiators

Collections 360 → Ask → Next Best Action, with governance in every layer. Every output shows its working: SQL, lineage, citations, reasons and an audit ID.



Layer 1

Trust score

Each golden record carries a trust score from match confidence, freshness and failed checks. Low trust makes decisions ask the agent to verify first.



Layer 2

Ask over governed metrics

The LLM fills a structured query over metrics defined once. Our code compiles the SQL, so roll rate means the same thing every time.



Layer 3

Affordability-first NBA

Risk plus ability to pay: payday detection, monthly capacity and a plan the customer can actually keep.



Layer 3

Live re-decisioning

When a call ends, features update, the model re-scores and the action changes in seconds, fully audited.



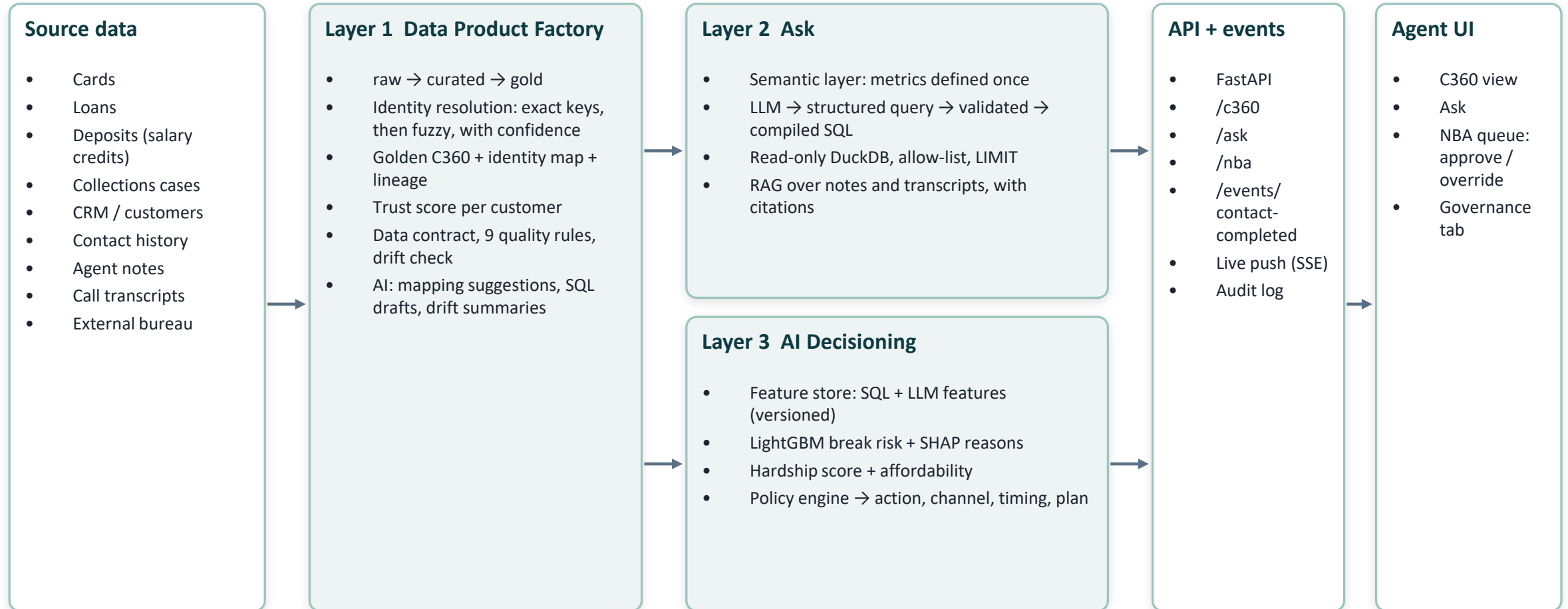
Oversight

Override intelligence

Reason-coded overrides and tracked outcomes become proposed policy changes that a human approves.

Most systems decide who to chase. Ours decides what each customer can afford, and learns from every human override.

Architecture: from nine sources to one fair decision

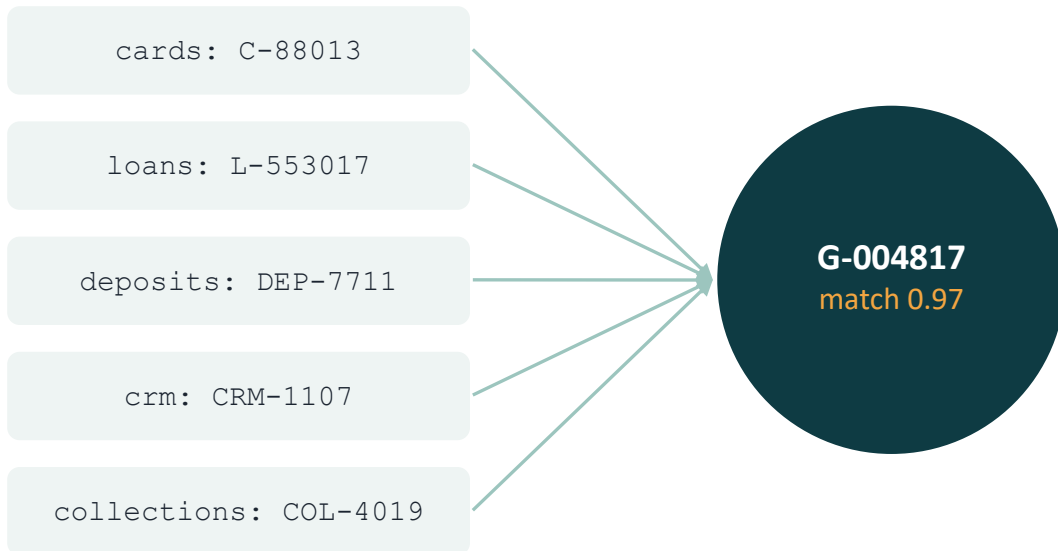


Governance in every layer: data contract · PII redaction · access by role · no protected attributes · hardship circuit breaker · explanations · human approve / override · audit log

Layer 1: one customer, one ID, one trust score



Identity resolution: exact keys first, then fuzzy matching, every link scored



Matches below 0.80 go to a manual review list. Source IDs are kept as lineage, so a wrong merge can be undone.



Trust score

$\text{trust} = \text{match confidence} \times \text{freshness} \times (1 - \text{share of failed checks on decision-critical fields})$

R. Mitchell $0.97 \times 1.00 \times 0.97 = 0.94$ **high trust**

V. Chen $0.84 \times 1.00 \times 0.97 = 0.81$ **verify identity first**

Below 0.85 the NBA tells the agent to verify identity before discussing balances. Below 0.60 no automated recommendation is made.

Data contract for gold.c360

Owner + purpose

21 columns, PII class each

9 quality rules

Allowed / prohibited uses

Drift check every run

AI-drafted SQL, tests decide

Layer 2: Ask, with the working shown

Which customers are most likely to break a promise to pay this week?

Understood as (click a chip to edit it)

open promises

due 28 Sep - 4 Oct

sort: break risk, high first

top 8

8 open promises are likely to break, with \$12,990 at risk.

Customer	Product	Promise	Due	Break risk
R. Mitchell	Loan + card	\$800	2 Oct	0.81
S. Tremblay	Card	\$2,250	30 Sep	0.78
A. Nguyen	Personal loan	\$1,900	1 Oct	0.74

```
SELECT c.display_name, p.product, p.ptp_amount, p.ptp_due_date, s.break_prob
FROM gold.promises_to_pay p JOIN gold.model_scores s USING (golden_id)
JOIN gold.c360 c USING (golden_id)
WHERE p.ptp_status = 'open' AND p.ptp_due_date BETWEEN '2026-09-28' AND '2026-10-04'
ORDER BY s.break_prob DESC LIMIT 8
```

Data products used: gold.promises_to_pay, gold.model_scores, gold.c360

Illustrative screen built from our mock data

How it works

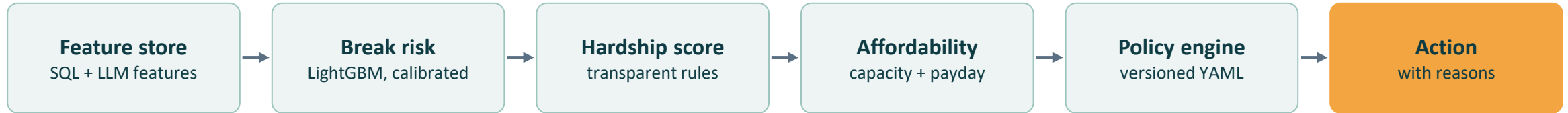
1. Parse: the LLM maps the question to {metric, dimensions, filters, time window, sort, limit}.
2. Validate against metrics.yaml and the column allow-list.
3. Compile: our code writes the SQL. For governed metrics the LLM never writes raw SQL.
4. Execute on read-only DuckDB, LIMIT enforced.
5. Explain: the LLM summarises only the returned rows. RAG over notes and transcripts adds cited quotes.



"What is the SIN of G-004817?"

Refused: direct personal identifiers are not available through Ask. Also refused: protected attributes, write operations, off-topic questions, prompt injection.

Layer 3: affordability-first Next Best Action



Policy: how risk, hardship and affordability combine

Hardship	Affordability	Risk	Action
Severe	any	any	Specialist referral, collection pressure blocked
Possible	ratio < 0.3	any	Agent review, offer a reduced plan
Clear	ratio ≥ 1	high	Call after payday, catch-up payment
Clear	0.3 to 1	any	Payment plan timed to payday
Clear	ratio ≥ 1	low	Light reminder on the best-responding channel

Always constrained by consent, permitted contact hours and the trust score. Thresholds are business-owned policy, not model output.

NBA-0001 · R. Mitchell

Specialist referral, collection pressure blocked

Top reasons

- No payroll deposit for 29 days
- Said shifts were cut at the plant in August
- One promise broken in the last 90 days

What would change it

If payroll arrives this week: risk 0.81 → 0.42 and the action becomes a reminder.

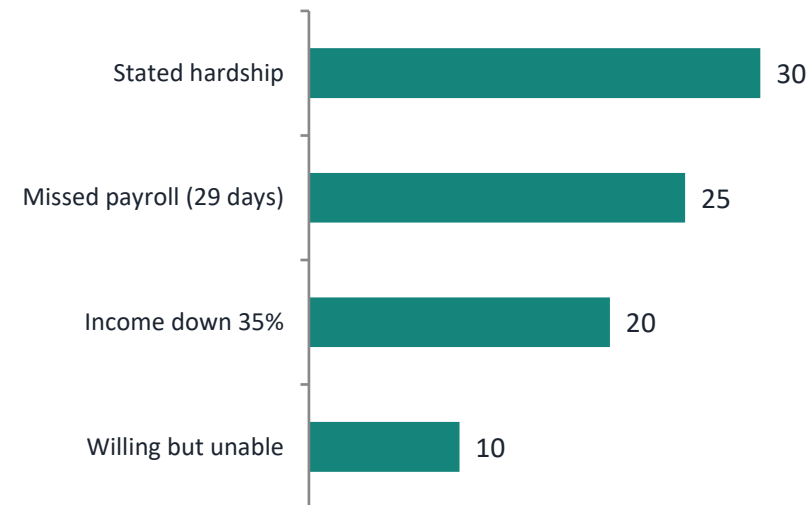
Approve

Override (reason required)

Hardship score: transparent rules, not a black box

Signal	Source	Rule	Points
Stated hardship	LLM, with quoted evidence	high / moderate severity	30 / 15
Missed payroll	Salary credits	no credit for > 1.5 × pay cycle	25
Income drop	Salary credits	last 60 days vs prior 90 down > 30%	20
Willing but unable	LLM intent + promises	intent ≥ 0.5 and a broken promise	10
Cash stress	Deposits	≥ 5 overdrawn days in 30	10
Credit stress	Bureau + cards	score −40 in 90 days, or utilisation > 90% rising	5 each
Vulnerability	LLM, with quoted evidence	illness, bereavement, distress, cognitive or language difficulty	always severe

Worked example: R. Mitchell



85 / 100

Severe: routed to a hardship specialist. The explanation cites the call quote.

0-29 Clear

30-59 Possible: agent review

60+ Severe: specialist only

Weights are versioned policy owned by the business and checked against hand-labelled cases. Health or bereavement details only ever protect the customer, are stored as a category, and protected attributes are never inputs.

Affordability: what can they pay, and when?

1

Pay cycle

Median gap between salary credits: weekly, biweekly or monthly. Next payday predicted.

2

Income

Typical pay × pays per month. After an income shock, use the latest pay.

3

Obligations

Loan instalments + card minimum payments.

4

Essentials

Policy allowance of 60% of income (configurable, labelled assumption).

5

Capacity

$C = \text{income} - \text{obligations} - \text{essentials}$. Buffer B = deposits above \$500.

6

Decide

$\text{ratio} = (3 \times C + B) \div \text{arrears}$. ≥ 1 catch-up · 0.3-1 staged plan · < 0.3 or $C \leq 0$ hardship pathway.

R. Mitchell

Shifts cut, payroll late

Income: \$2,058 / month (latest \$950 biweekly)

Obligations: \$830

Essentials: \$1,235

-\$7

capacity / month

Hardship pathway: specialist, reduced payment or payment holiday, no collection calls.

M. Roy

Stable salary, has savings

Income: \$5,800 / month

Obligations: \$1,550

Essentials: \$3,480

1.05

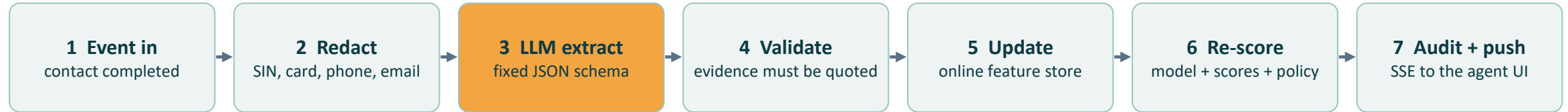
affordability ratio

Call the day after payday: \$950 from savings, then \$616 / month for 4 months.

*Plans use 80% of capacity as a safety margin and fall due the day after the predicted payday.
Confidence = pay-cycle regularity × trust score.*

Both are high-risk. The actions are opposite, and the system shows why.

Live re-decisioning: the action changes when the call ends



Before · batch run 06:10

Hardship
45 possible

Break risk
0.58

Action
Call after payday

After · call ends 14:12, decision 3 s later

Hardship
85 severe

Break risk
0.81

Action
Specialist referral, pressure blocked

Evidence: "I still have no full shifts. I can do \$800 by October 2nd." Audit: features_updated, rescored, decision_changed, hardship_routed.

Production: Kafka / Event Hubs → stream processor → online feature store (Redis / Feast) → model endpoint. Same logic, different runtime.



The LLM never decides

It turns text into structured facts. Code does the rest.



Evidence or it didn't happen

A claim without a verbatim quote is dropped.



Safe failure

On timeout or invalid output: keep the previous decision, flag for review.



Idempotent and fast

Same event_id changes nothing. About 2-4 s end to end.

How the AI is powered: the LLM proposes, code decides

Job	Engine	Why
Extract features from notes and transcripts	Hosted LLM, fast tier, JSON schema output	High volume, cheap, structured
Question → semantic query (Ask)	Stronger LLM	Reasoning over metrics and time windows
RAG answers, summaries, override insights	Stronger LLM	Writing quality; numbers always from SQL
Embeddings for RAG	Local sentence-transformers	Customer text never leaves for indexing
Promise-break risk	LightGBM + calibration + SHAP	Accurate on tables, explainable, real probabilities
Channel and timing	Response rates by channel and hour	Simple, transparent, enough signal
Decision policy	Deterministic rules in versioned YAML	Auditable, owned by the business



Controls around every model

- One client wrapper: swap providers, or run a local Llama / Qwen where data residency matters
- PII redacted before every LLM call
- Cache by text hash + prompt version; spend cap; limited retries
- Every LLM feature has a prompt version and an evaluation against hand labels
- Strictest target: recall on severe hardship
- Model trained on a time split, calibrated, promoted only after human approval

Governance you can see in the product



Data (Layer 1)

- Published data contract
- 9 quality rules, drift check
- PII class per column, lineage
- Trust score per customer



Ask (Layer 2)

- SELECT-only, allow-list, LIMIT
- PII and protected columns blocked
- SQL and sources on every answer
- Refusals with reasons



Decisions (Layer 3)

- No protected attributes, proven by a test
- Hardship circuit breaker enforced in the API
- Consent and contact hours respected
- Reasons + counterfactual per decision



Human oversight

- Approve or override, reason code required
- Specialist-only hardship approvals
- Audit log of every event
- Outcomes compared across segments



Override intelligence

1 Override with reason code

2 Track the outcome

3 Spot the pattern

4 Propose a policy change

5 Human approves

"Agents changed call to payment plan for self-employed customers in 7 of 10 cases, and those plans were kept more often. Proposed: payment plan by default for this group. Awaiting approval."

Illustrative insight. The model never retrains itself silently.

Implementation plan: 36 hours, two people, contract-first

	Day 1 morning	Day 1 afternoon	Day 1 evening	Day 2 morning	Day 2 afternoon
Person A pipeline, decisioning, API	Load data; raw → curated; identity resolution	Golden C360, trust score, contract + DQ report	Feature store; LLM features + evaluation	Risk model, hardship, affordability, policy, live re-decision	Integrate, audit log, freeze at 17:00, demo video
Person B Ask, RAG, agent UI	Semantic layer, Ask parser, SQL guard	RAG with citations; Ask UI; C360 view	Benchmark ≥ 85%; switch to real data	NBA queue, decision-changed card, override flow	Governance tab, polish, final deck

Already done during the design phase

API contract: 11 endpoints	Data contract: 9 quality rules	Ask schema allow-list
Mock data: 10 customers	Architecture document	AI-agent workflow + reporting



How we build

- Contract first: API, schema and mocks let UI and backend start in parallel at hour one
- AI agents (Claude Code, Antigravity, ChatGPT) build in owned folders and post progress reports to the repo
- A lead agent reviews the reports, checks the work and issues the next instructions

How we will prove it works

$\geq 85\%$

Ask benchmark pass rate

20+ questions on real data

100%

**must-refuse questions
refused**

PII, protected, writes, injection

≥ 0.90

recall on severe hardship

vs hand-labelled notes (target)

AUC

+ calibration plot

break-risk model, time-split test

9 rules

data quality checked

on every pipeline run

These are targets we will measure and report honestly, including misses.

MVP: must ship

- C360 with identity resolution, trust score, contract and DQ report
- Ask over governed metrics, SQL shown, refusals, RAG citations
- NBA with risk, hardship and affordability, plain-English reasons
- Approve / override with reason, specialist-only hardship, audit log
- Live re-decisioning on a new transcript

Stretch: cut first if time is short, in this order

1. Counterfactual explanations
2. Override insights (keep the reason codes)
3. Free-form SQL fallback in Ask
4. Channel and timing model beyond response rates

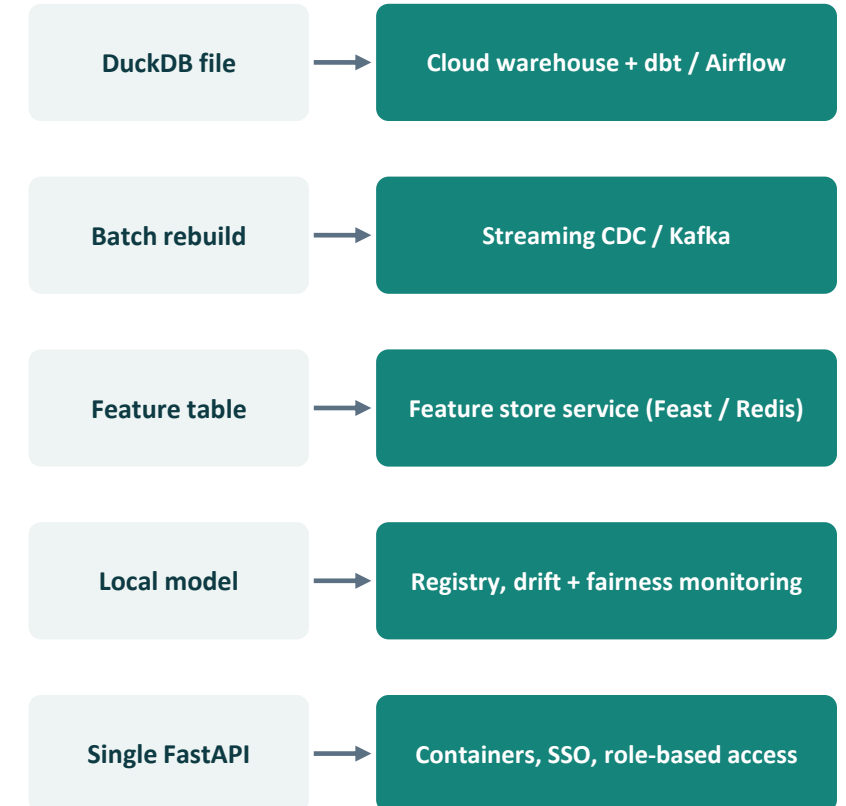
Business value and the path to scale

KPI	Assumed baseline	Mechanism	Assumed target
Promise-kept rate	60%	Affordable plans, payday timing	+5 pts
Roll rate	current level	Early, affordable plans before a roll	lower
Contact efficiency	1 in 5 attempts	Best channel and hour	+15% relative
Agent handle time	10 min / account	C360 + pre-built reasons	-2 min (~20%)
Analyst question	days	Ask with visible SQL	minutes

≈ 20

agent-equivalents of capacity freed in a 100-agent team if handle time drops from 10 to 8 minutes: the same workload with about 80 agents.

Prototype → production



All baselines and targets are our assumptions to be validated, not bank figures.

The three-minute demo

- 1 Open R. Mitchell's Collections 360: nine sources, one ID, trust 0.94.
- 2 Ask "who will break a promise this week?" and see the ranked list with its SQL.
- 3 Ask "who mentioned job loss?" and see cited call quotes. Ask for his SIN and see a refusal.
- 4 Drop in his new call transcript: hardship 45 → 85, the action changes to a specialist referral.
- 5 Show M. Roy: high risk too, but he can afford it, so a call the day after payday.
- 6 Override another case with a reason. The Governance tab shows the audit trail and fairness checks.

Resolve360: stop chasing, start resolving.